

Permutations with Repetition: A Password Cracker

Imagine you have received a sensitive encrypted file from a recently deceased journalist. In their final message, the journalist told you the file contains records of tax evasion by a nefarious trillionaire. They didn't have the password to decrypt the file, but they did know that it is exactly four characters long; also, the possible characters are the numbers 2, 4, and 8 and the letters J, P, and B. These characters can appear more than once. For example, possible passwords are JPB2, JJJJ, and 2442.

To generate a list of all possible four-character passwords based on this information, you want to obtain all possible four-element permutations with repetition of the set {J, P, B, 2, 4, 8}. Each of the four characters in the password can be one of the six possible characters, making $6 \times 6 \times 6 \times 6$, or 6^4 , or 1,296 possible permutations. We want to generate the permutations of {J, P, B, 2, 4, 8}, and not the combinations, because the ordering matters; JPB2 is a different password from B2JP.

Let's ask the three recursive algorithm questions about our permutations function. Instead of k , we'll use the more descriptive name `permLength`:

What is the base case? A `permLength` argument of 0, meaning a permutation zero characters long, signals that the `prefix` argument now contains the complete permutation and so `prefix` should be returned in an array.

What argument is passed to the recursive function call? The `chars` string of the characters to get permutations of, a `permLength` argument that begins as the length of `chars`, and a `prefix` argument that begins as the blank string. Recursive calls decrement the `permLength` argument while appending a character from `chars` to the `prefix` argument.

How does this argument become closer to the base case?

Eventually, the `permLength` argument decrements to 0.

The algorithm for recursive permutations with repetition is implemented in *permutationsWithRepetition.py*: